

techniques have become popular only in the last decade, they are basically variants of “top-down parsing languages” described in the early 1970.

3.11 Exercises

The main goal of these exercises is to familiarize yourself with regular expressions, automata, grammars, the `fslex` lexer generator and the `fsyacc` parser generator.

Exercise 3.1 Do exercises 1.2 and 1.3 in Mogensen’s book [12].

Exercise 3.2 Write a regular expression that recognizes all sequences consisting of a and b where two a ’s are always separated by at least one b . For instance, these four strings are legal: b , a , ba , $ababbbaba$; but these two strings are illegal: aa , $babaa$.

Construct the corresponding NFA. Try to find a DFA corresponding to the NFA.

Exercise 3.3 Write out the rightmost derivation of the string below from the expression grammar at the end of Sect. 3.6.5, corresponding to `ExprPar.fsy`. Take note of the sequence of grammar rules (A–I) used.

```
let z = (17) in z + 2 * 3 end EOF
```

Exercise 3.4 Draw the above derivation as a tree.

Exercise 3.5 Get `expr.zip` from the book homepage and unpack it. Using a command prompt, generate (1) the lexer and (2) the parser for expressions by running `fslex` and `fsyacc`; then (3) load the expression abstract syntax, the lexer and parser modules, and the expression interpreter and compilers, into an interactive F# session (`fsharp`):

```
fslex --unicode ExprLex.fsl
fsyacc --module ExprPar ExprPar.fsy
fsharp -r FSharp.PowerPack.dll Absyn.fs ExprPar.fs ExprLex.fs ^
    Parse.fs
```

Now try the parser on several example expressions, both well-formed and ill-formed ones, such as these, and some of your own invention:

```
open Parse;;
fromString "1 + 2 * 3";;
fromString "1 - 2 - 3";;
fromString "1 + -2";;
fromString "x++";;
fromString "1 + 1.2";;
fromString "1 + ";;
fromString "let z = (17) in z + 2 * 3 end";;
fromString "let z = 17) in z + 2 * 3 end";;
fromString "let in = (17) in z + 2 * 3 end";;
fromString "1 + let x=5 in let y=7+x in y+y end + x end";;
```

Exercise 3.6 Use the expression parser from `Parse.fs` and the compiler `scomp` (from expressions to stack machine instructions) and the associated datatypes from `Expr.fs`, to define a function `compString : string -> sinstr list` that parses a string as an expression and compiles it to stack machine code.

Exercise 3.7 Extend the expression language abstract syntax and the lexer and parser specifications with conditional expressions. The abstract syntax should be `If(e1, e2, e3)`, so modify file `Absyn.fs` as well as `ExprLex.fsl` and file `ExprPar.fsy`. The concrete syntax may be the keyword-laden F#/ML-style:

```
if e1 then e2 else e3
```

or the more light-weight C/C++/Java/C#-style:

```
e1 ? e2 : e3
```

Some documentation for `fslex` and `fsyacc` is found in this chapter and in *Expert F#* [17].

Exercise 3.8 Consider the parser generated from `ExprPar.fsy`, and determine what steps are taken during the parsing of this string:

```
let z = (17) in z + 2 * 3 end EOF
```

For each step, show the remaining input, the parse stack, and the action (shift, reduce, or goto) performed. You will need a printout of the parser states and their transitions in `ExprPar.fsyacc.output` to do this exercise. Sanity check: the sequence of reduce action rule numbers in the parse should be the exact reverse of that found in the derivation in Exercise 3.3.

Exercise 3.9 Files in the subdirectory `Usql/` contain abstract syntax abstract syntax (file `Absyn.fs`), an informal grammar (file `grammar.txt`), a lexer specification (`UsqlLex.fsl`) and a parser specification (`UsqlPar.fsy`) for micro-SQL, a small subset of the SQL database query language.

Extend micro-SQL to cover a larger class of SQL `SELECT` statements. Look at the examples below and decide your level of ambition. You should not need to modify file `Parse.fs`. Don't forget to write some examples in concrete syntax to show that your parser can parse them.

For instance, to permit an optional `WHERE` clause, you may add one more components to the `Select` constructor:

```
type stmt =
| Select of expr list           (* fields are expressions *)
          * string list        (* FROM ... *)
          * expr option        (* optional WHERE clause *)
```

so that `SELECT...FROM...WHERE...` gives `Select(..., ..., SOME ...)`, and `SELECT...FROM...` gives `Select(..., ..., NONE)`.

Mads Tofte, Bob Harper, and David MacQueen, and OCaml [8, 15] by Xavier Leroy and Damien Doligez.

Whereas these languages have so-called *strict* or *eager* evaluation – function arguments are evaluated before the function is called – another subfamily is made up of the so-called *non-strict* or *lazy* functional languages, including SASL and its successor Miranda [19] both developed by David Turner, Lazy ML [1, 5] developed by Lennart Augustsson and Thomas Johnsson, and Haskell [16] where driving forces are Simon Peyton Jones, John Hughes, Paul Hudak, and Phil Wadler. All the statically typed languages are statically scoped as well.

Probably the first publication about type checking in a compiler is Peter Naur’s description [13] of the Algol 60 compilers developed at Regnecentralen in Copenhagen.

More general forms of static analysis or static checking have been studied under the name of data flow analysis [7], or control flow analysis, or abstract interpretation [2], and in much subsequent work. See Nielson, Nielson and Hankin [14] for much more information.

4.12 Exercises

The goal of these exercises is to understand the evaluation of a simple first-order functional language, and how explicit types can be stated and checked.

Exercise 4.1 Get archive `fun.zip` from the homepage and unpack to directory `Fun`. It contains lexer and parser specifications and interpreter for a small first-order functional language. Generate and compile the lexer and parser as described in `README.TXT`; parse and run some example programs with `ParseAndRun.fs`.

Exercise 4.2 Write more example programs in the functional language, and test them in the same way as in Exercise 4.1:

- Compute the sum of the numbers from 1000 down to 1. Do this by defining a function `sum n` that computes the sum $n + (n - 1) + \dots + 2 + 1$. (Use straightforward summation, no clever tricks).
- Compute the number 3^8 , that is, 3 raised to the power 8. Again, use a recursive function.
- Compute $3^0 + 3^1 + \dots + 3^{10} + 3^{11}$, using a recursive function (or two, if you prefer).
- Compute $1^8 + 2^8 + \dots + 10^8$, again using a recursive function (or two).

Exercise 4.3 For simplicity, the current implementation of the functional language requires all functions to take exactly one argument. This seriously limits the programs that can be written in the language (at least it limits what that can be written without excessive cleverness and complications).

Modify the language to allow functions to take one or more arguments. Start by modifying the abstract syntax in `Absyn.fs` to permit a list of parameter names in `Letfun` and a list of argument expressions in `Call`.

Then modify the `eval` interpreter in file `Fun.fs` to work for the new abstract syntax. You must modify the closure representation to accommodate a list of parameters. Also, modify the `Letfun` and `Call` clauses of the interpreter. You will need a way to zip together a list of variable names and a list of variable values, to get an environment in the form of an association list; so function `List.zip` might be useful.

Exercise 4.4 In continuation of Exercise 4.3, modify the parser specification to accept a language where functions may take any (non-zero) number of arguments. The resulting parser should permit function declarations such as these:

```
let pow x n = if n=0 then 1 else x * pow x (n-1) in pow 3 8 end

let max2 a b = if a<b then b else a
in let max3 a b c = max2 a (max2 b c)
  in max3 25 6 62 end
end
```

You may want to define non-empty parameter lists and argument lists in analogy with the `Names1` nonterminal from `Usql/UsqlPar.fsy`, except that the parameters should not be separated by commas. Note that multi-argument applications such as `f a b` are already permitted by the existing grammar, but they would produce abstract syntax of the form `Call(Call(Var "f", Var "a"), Var "b")` which the `Fun.eval` function does not understand. You need to modify the `AppExpr` nonterminal and its semantic action to produce `Call(Var "f", [Var "a"; Var "b"])` instead.

Exercise 4.5 Extend the (untyped) functional language with infix operator “&&” meaning sequential logical “and” and infix operator “|” meaning sequential logical “or”, as in C, C++, Java, C#, or F#. Note that `e1 && e2` can be encoded as `if e1 then e2 else false` and that `e1 || e2` can be encoded as `if e1 then true else e2`. Hence you need only change the lexer and parser specifications, and make the new rules in the parser specification generate the appropriate abstract syntax. You need not change `Absyn.fs` or `Fun.fs`.

Exercise 4.6 Extend the abstract syntax, the concrete syntax, and the interpreter for the untyped functional language to handle tuple constructors `(...)` and component selectors `#i` (where the first component is `#1`):

```
type expr =
  | ...
  | Tup of expr list
  | Sel of int * expr
  | ...
```